

AchievementSystem

for Unity

Official Documentation

A lightweight, framework-agnostic achievements library for Unity games. Simple unlocks, progress counters with shared-key tiers, and AND/OR composite achievements — all driven by ScriptableObject data.

Version 1.0.0 · Unity 6000.3+ · also works on 2021.3 LTS+

Publisher: ZorCeDev · zorcedev@gmail.com

Table of Contents

1. Overview
2. Requirements
3. Installation & Setup
 - Path A — Without Zenject (Standalone)
 - Path B — With Zenject / Extenject
4. Achievement Types
 - Simple, Progress (Tiers), Composite, Hidden
5. The Default Achievement Popup
6. Platform Sync
7. Localization
8. Addressables (Deferred Icon Loading)
9. Editor Tooling — Achievement Manager
10. Architecture Overview
11. Script Reference
 - `IAchievementService`
 - `IAchievementDefinition`
 - `IAchievementRepository`
 - `IPlatformAchievementSync`
 - `IAchievementLocalizationProvider`
12. Custom Repository Guide
13. Render Pipeline Compatibility
14. Troubleshooting
15. Support & Contact

| 1. Overview

AchievementSystem is a lightweight, framework-agnostic achievements library for Unity. It provides Simple, Progress (with shared-key tiers), and Composite (AND/OR) achievements, backed entirely by ScriptableObject data, with no mandatory UI, dependency injection framework, or platform SDK.

Key Features

- **One-liner API** — unlock achievements and add progress with a single method call.
- **Every achievement shape** — Simple, Progress with Bronze/Silver/Gold-style tiers, and AND/OR Composite/meta-achievements.
- **Ready-to-use unlock popup** — a drag-in prefab that queues simultaneous unlocks instead of overlapping them.
- **Auto-wired platform sync** — attach your own Steam/Google Play/Game Center adapter and it is discovered automatically.
- **Optional runtime localization** — zero dependencies, with automatic fallback to the default text.
- **Optional Zenject/Extenject and Addressables integrations** — both auto-detected, and both compile to nothing when the corresponding package is absent.
- **Editor tooling** — one window validates achievement data and simulates every unlock path in Play Mode.

2. Requirements

Requirement	Version / Notes
Unity	6000.3 or newer (also works on 2021.3 LTS and later)
TextMeshPro	Bundled with com.unity.ugui 2.0.0+. Import TMP Essential Resources once, as with any TextMeshPro project.
Zenject / Extenject	Optional. Required only if you use the Zenject installer (see Section 3, Path B).
Addressables	Optional. Required only for deferred icon loading (see Section 8).
Steamworks.NET	Optional. Required only if you enable the Steam platform adapter.
Google Play Games Plugin	Optional. Required only if you enable the Google Play platform adapter.

3. Installation & Setup

Step 1 — Import the package

In Unity, go to **Assets** → **Import Package** → **Custom Package** and select **AchievementSystem.unitypackage**. Import all files.

Step 2 — Create your achievements

Right-click in the Project window:

Create → Achievements → Achievement Definition

Fill in **Id** (unique, used in code), **Display Name**, **Description**, **Category**, and **Type**. Repeat once per achievement.

Step 3 — Create the database

Create → Achievements → Achievement Database

Drag every Achievement Definition asset into its list.

Path A — Without Zenject (Standalone)

1. Create an empty GameObject in your first scene and name it AchievementSystem.
2. Attach the **AchievementBootstrapper** component.
3. Assign your Achievement Database asset to the **Database** field.
4. Done. The bootstrapper calls DontDestroyOnLoad automatically.

Access the service from anywhere in your code:

```
using Achievements.Unity;  
  
AchievementLocator.Service.Unlock("first_death");  
AchievementLocator.Service.AddProgress("enemies_killed", 1);
```

Path B — With Zenject / Extenject

This path is only available once Extenject is installed in your project — the integration compiles to nothing until then.

1. Create an installer asset: `Create → Achievements → Installer`
2. Assign your Achievement Database (and optionally Unlocked Channel, Progress Channel, Localization Table).
3. Add the installer to your **ProjectContext** or **SceneContext**.
4. Inject `IAchievementService` wherever you need it.

Use **AchievementInstaller** instead of **AchievementBootstrapper** — not both.

```
public sealed class LevelManager : MonoBehaviour
{
    [Inject] private IAchievementService _achievements;

    private void OnEnemyKilled()
    {
        _achievements.AddProgress("enemies_killed", 1);
    }
}
```

| 4. Achievement Types

Simple

One-shot unlocks such as a first login or a story milestone. Call `Unlock(id)` whenever your gameplay code decides the condition has been met.

Progress (with Tiers)

A shared counter with a threshold. Create several Progress definitions that use the same Progress Key with different thresholds (for example 10 / 50 / 100) to get Bronze/Silver/Gold-style tiers from a single counter. One call to `AddProgress` can unlock more than one tier at once if the count jumps far enough.

Composite

"Unlock all achievements in category X" or "unlock these specific achievements" style meta-achievements, combined with **All** or **Any** logic, driven by Required Achievement Ids and/or Required Category.

Hidden

A presentation flag only. The achievement's name and description remain present on the asset; your UI decides what to display for a hidden, locked achievement.

| 5. The Default Achievement Popup

`Prefabs/AchievementUnlockedPopup.prefab` is a ready-to-use slide-in notification.

1. Drag the prefab under any Canvas in your scene.
2. Assign your Unlocked Channel asset (the same one on AchievementBootstrapper) to its **Unlocked Channel** field.
3. Restyle freely — background image, fonts, colors, size, or the slide distance/duration fields.

The popup automatically queues simultaneous unlocks so they slide in one after another instead of overlapping, and shows the achievement's localized display name if a Localization Table is configured (see Section 7).

6. Platform Sync

Implement `IPlatformAchievementSync` as a `MonoBehaviour` and attach it to the same `GameObject` as `AchievementBootstrapper`. It is discovered automatically at startup — no code changes to the bootstrapper are required. A failing adapter never blocks local unlocking or the other adapters; each call is isolated.

Included Adapters (Unity/PlatformSync/)

Adapter	Dependency	Notes
GameCenterAchievementSync	None	Live by default. Built entirely on Unity's own Social API.
SteamAchievementSync	Steamworks.NET	Reference implementation, provided commented out until the SDK is installed.
GooglePlayAchievementSync	Google Play Games Plugin	Reference implementation, provided commented out until the plugin is installed.

7. Localization

Achievement text can be localized without touching gameplay code.

1. Create a table: `Create → Achievements → Localization Table`, add one entry per achievement

and locale (for example `first_death` / `es` / “Primera Muerte”).

2. Assign the table to AchievementBootstrapper's **Localization Table** field.
3. Switch languages at runtime with `AchievementLocator.Service.SetLocale("es")`.

If no table is assigned, or the active locale has no entry for an achievement, the system falls back to the table's default locale and finally to the achievement's own Display Name/Description. The `LocaleChanged` event fires after `SetLocale`, so UI can refresh already-visible text.

| 8. Addressables (Deferred Icon Loading)

For achievement sets large enough that memory matters, icon sprites can be loaded on demand via Addressables instead of kept resident through direct references. This is only available once the Addressables package is installed; otherwise the integration compiles to nothing.

1. Create a set: `Create → Achievements → Addressables → Icon Set`, add one entry per

achievement with its locked/unlocked `AssetReferenceSprite`.

2. Load on demand:

```
var loader = new AchievementIconAddressableLoader(iconSet);
var sprite = await loader.LoadUnlockedIconAsync("first_death");

// later, when the UI showing it goes away:
loader.ReleaseIcon("first_death", locked: false);
```

This is entirely separate from Achievement Definition's direct Locked Icon/Unlocked Icon fields — use whichever fits your project, or mix both.

| 9. Editor Tooling — Achievement Manager

Window → Achievements → Achievement Manager

Validation

Always available. Flags duplicate or missing Ids across every Achievement Definition in your project, so typos get caught before they ship.

Simulation

Play Mode only. One row per achievement with Unlock, +1 Progress, and Reset buttons, plus a global Reset All — lets you walk through every achievement path with clicks, no gameplay code required.

10. Architecture Overview

```
Your Game Code
    v IAchievementService (AchievementLocator.Service, or inject directly)
AchievementService
    |-- IAchievementRepository      -> PlayerPrefs (default) or JSON file
    |-- IPlatformAchievementSync    -> auto-discovered MonoBehaviours
    |-- IAchievementLocalizationProvider -> optional, falls back to definition text
    `-- events -> ScriptableObject Events (decouple UI from core)
```

Assembly Structure

Assembly	Contents
Achievements.Core	Models, interfaces, AchievementService (pure logic, no Unity dependency).
Achievements.Unity	ScriptableObjects, MonoBehaviours, repositories, locator.
Achievements.UI	Default, reusable UI components (the unlock popup).
Achievements.Zenject	Optional Zenject/Extenject installer. Compiles to nothing without Extenject.
Achievements.Addressables	Optional deferred icon loading. Compiles to nothing without Addressables.
Achievements.Editor	Achievement Manager window (Editor only).
Achievements.Tests	NUnit test suite (17 tests) for AchievementService.

11. Script Reference

IAchievementService

Member	Description
Unlock(string id)	Unlocks the achievement. Idempotent — calling it again is a no-op.
AddProgress(string key, int amount = 1)	Increments a progress counter and unlocks any Progress achievement whose threshold is reached.
SetProgress(string key, int value)	Sets a progress counter to an exact value.
IsUnlocked(string id)	Returns whether the achievement is currently unlocked.
GetProgress(string key)	Returns the current value of a progress counter.
GetDisplayName(string id) / GetDescription(string id)	Returns localized text for the active locale, with fallback to the definition's own text.
SetLocale(string locale)	Changes the active locale and raises LocaleChanged.
ResetAchievement / ResetProgress / ResetAll	Clears unlocked state and/or progress.
AchievementUnlocked, ProgressChanged, LocaleChanged	Events raised on the corresponding change.

IAchievementDefinition

Id, DisplayName, Description, Category, IsHidden, LockedIcon, UnlockedIcon, Type, ProgressKey, ProgressThreshold, CompositeLogic, RequiredAchievementIds, RequiredCategory.

IAchievementRepository

TryGetUnlocked(id, out unlockedAtUtc), SetUnlocked(id, unlockedAtUtc), ClearUnlocked(id), GetProgress(key), SetProgress(key, value), ClearProgress(key). Implement this to plug in your own save system.

IPlatformAchievementSync

PlatformId, NotifyUnlocked(id). Implement as a MonoBehaviour attached next to AchievementBootstrapper for automatic discovery (see Section 6).

IAchievementLocalizationProvider

CurrentLocale, SetLocale(locale), TryGetDisplayName(id, out displayName), TryGetDescription(id, out description). Implement this to source translations from Unity Localization, a remote CMS, or any other system.

| 12. Custom Repository Guide

Implement `IAchievementRepository` to plug in your own save system — cloud saves, a custom binary format, or anything else — and pass it into `AchievementService`'s constructor yourself instead of using `AchievementBootstrapper`.

```
IAchievementRepository repository = new MyCloudSaveRepository();  
var service = new AchievementService(definitions, repository);
```

The two included implementations, `PlayerPrefsAchievementRepository` and `JsonFileAchievementRepository`, store data unencrypted. This is normal for an offline single-player save, but worth a note in your own game's documentation if you enable platform sync, since a locally tampered unlock would sync as genuine.

| 13. Render Pipeline Compatibility

Built-in, URP, and HDRP all work without changes. The UI is plain UGUI (Image with the built-in default sprite) and TextMeshPro on a Screen Space Overlay canvas, which renders outside the render pipeline's camera stack. No custom shaders, materials, or particle systems are used anywhere in the package.

The only external requirement is TextMeshPro's own **Window > TextMeshPro > Import TMP Essential Resources**, which every TextMeshPro project needs regardless of this package.

14. Troubleshooting

Symptom	Likely Cause
AchievementLocator.Service throws on startup	AchievementBootstrapper has not run yet in this scene. Make sure it exists in the very first scene that loads.
Calling Unlock does nothing, no error	Check that the Id matches an Achievement Definition's Id exactly (case-sensitive) and is included in the assigned database.
Progress tiers all unlock at once	They likely share the same Progress Key by mistake — only intended for actual tiers of the same counter.
Achievement list/toast UI shows nothing	Confirm the same Unlocked/Progress Channel asset is assigned on both AchievementBootstrapper and the UI components.
Unlocks do not persist between sessions	Check AchievementBootstrapper's Repository Type field, and whether ResetAll is being called somewhere.

| 15. Support & Contact

For questions, bug reports, or feature requests after purchase, contact:

zorcedev@gmail.com

Thank you for using AchievementSystem.

Publisher: ZorCeDev